## 4.6 SORTING; BUBBLE SORT

Let A be a list of $n$ numbers. *Sorting* A refers to the operation of rearranging the elements of A so they are in increasing order, i.e., so that,

$$A[1] < A[2] < A[3] < \ldots < A[N]$$

For example, suppose A originaily is the list

8, 4, 19, 2, 7, 13, 5, 16

After sorting, A is the list

2, 4, 5, 7, 8, 13, 16, 19

Sorting may seem to be a trivial task. Actually, sorting efficiently may be quite complicated. In fact, there are many, many different sorting algorithms; some of these algorithms are discussed in Chapter 9. Here we present and discuss a very simple sorting algorithm known as the *bubble sort*.

*Remark:* The above definition of sorting refers to arranging numerical data in increasing order; this restriction is only for notational convenience. Clearly, sorting may also mean arranging numerical data in decreasing order or arranging non-numerical data in alphabetical order. Actually, A is frequently a file of records, and sorting A refers to rearranging the records of A so that the values of a given key are ordered.

### Bubble Sort

Suppose the list of numbers A[l], A[2], ... ,A[N] is in memory. The bubble sort algorithm works as follows:

Step 1. Compare A[1] and A[2] and arrange them in the desired order, so that A[1] < A[2]. Then compare A[2] and A[3] and arrange them so that A[2] < A[3]. Then compare A[3] and A[4] and arrange them so that A[3] < A[4]. Continue until we compare A[N – 1] with A[N] and arrange them so that A[N – 1] < A[N].

Observe that Step 1 involves $n - 1$ comparisons. (During Step 1, the largest element is "bubbled up" to the $n$th position or "sinks" to the $n$th position.) When Step 1 is completed, A[N] will contain the largest element.

Step 2. Repeat Step 1 with one less comparison; that is, now we stop after we compare and possibly rearrange A[N – 2] and A[N – 1]. (Step 2 involves N – 2 comparisons and, when Step 2 is completed, the second largest element will occupy A[N – 1].)

Step 3. Repeat Step 1 with two fewer comparisons; that is, we stop after we compare and possibly rearrange A[N – 3] and A[N – 2].

. .....

. .....

. .....

*Step N – 1.* Compare A[1] with A[2] and arrange them so that A[1] < A[2].

After $n - 1$ steps, the list will be sorted in increasing order.

The process of sequentially traversing through all or part of a list is frequently called a "pass," so each of the above steps is called a pass. Accordingly, the bubble sort algorithm requires $n - 1$ passes, where $n$ is the number of input items,

## Example 4.7

Suppose the following numbers are stored in an array A:

32, 51, 27, 85, 66, 23, 13, 57

We apply the bubble sort to the array A. We discuss each pass separately.

Pass 1. We have the following comparisons:

**(a)** Compare $A_1$ and $A_2$. Since 32 < 51, the list is not altered.

**(b)** Compare $A_2$ and $A_3$. Since 51 > 27, interchange 51 and 27 as follows:

32, (27, (51, 85, 66, 23, 13, 57

**(c)** Compare $A_3$ and $A_4$. Since 51 < 85, the list is not altered.

**(d)** Compare $A_4$ and $A_5$. Since 85 > 66, interchange 85 and 86 as follows:

32, 27, 51, (66, (85, 23, 13, 57

**(e)** Compare $A_5$ and $A_6$. Since 85 > 23, interchange 85 and 23 as follows:

32, 27, 51, 66, (23, (85, 13, 57

**(f)** Compare $A_6$ and $A_7$. Since 85 > 13, interchange 85 and 13 to yield:

32, 27, 51, 66, 23, (13, (85, 57

**(g)** Compare $A_7$ and $A_8$. Since 85 > 57, interchange 85 and 51 to yield:

32, 27, 51, 66, 23, 13, (57, (85

At the end of this first pass, the largest number, 85, has moved to the last position. However, the rest of the numbers are not sorted, even though some of them have changed their positions.

For the remainder of the passes, we show only the interchanges.

Pass 2. (27, 33) 51, 66, 23, 13, 57, 85
27, 33, 51, (23, 66) 13, 57, 85
27, 33, 51, 23, (13, 66) 57, 85
27, 33, 51, 23, 13, (57, 66) 85

At the end of Pass 2, the second largest number, 66, has moved its way down to the next-to-last position.

Pass 3. 27, 33, (23, 51) 13, 57, 66, 85
27, 33, 23, (13, 51) 57, 66, 85

Pass 4. 27, (23, 33) 13, 51, 57, 66, 85
27, 23, (13, 33) 51, 57, 66, 85

Pass 5. (23, 27) 13, 33, 51, 57, 66, 85
23, (13, 27) 33, 51, 57, 66, 85

Pass 6. (13, 23) 27, 33, 51, 57, 66, 85

Pass 6 actually has two comparisons, $A_1$ with $A_2$ and $A_2$ and $A_3$. The second comparison does not involve an interchange.

Pass 7. Finally, $A_1$ is compared with $A_2$. Since $13 < 23$, no interchange takes place.

Since the list has 8 elements; it is sorted after the seventh pass. (Observe that in this example, the list was actually sorted after the sixth pass. This condition is discussed at the end of the section.)

We now formally state the bubble sort algorithm.

**Algorithm 4.4:** (Bubble Sort) BUBBLE(DATA, N)
Here DATA is an array with N elements. This algorithm sorts the elements in DATA.

1. Repeat Steps 2 and 3 for K = 1 to N – 1.
2. Set PTR := 1. [Initializes pass pointer PTR.]
3. Repeat while PTR ≤ N – K: [Executes pass.]
   (a) If DATA[PTR] > DATA[PTR + 1], then:
   Interchange DATA[PTR] and DATA[PTR + 1].
   [End of If structure.]
   (b) Set PTR := PTR + 1.
   [End of inner loop.]
   [End of Step 1 outer loop.]
4. Exit.

Observe that there is an inner loop which is controlled by the variable PTR, and the loop is contained in an outer loop which is controlled by an index K. Also observe that PTR is used as a subscript but K is not used as a subscript, but rather as a counter.

## Complexity of the Bubble Sort Algorithm

Traditionally, the time for a sorting algorithm is measured in terms of the number of comparisons. The number $f(n)$ of comparisons in the bubble sort is easily computed. Specifically, there are $n - 1$ comparisons during the first pass, which places the largest element in the last position; there are $n - 2$ comparisons in the second step, which places the second largest element in the next-to-last position; and so on. Thus

$$f(n) = (n-1) + (n-2) + \ldots + 2 + 1 = \frac{n(n-1)}{2} = \frac{n^2}{2} + O(n) = O(n^2)$$

In other words, the time required to execute the bubble sort algorithm is proportional to $n^2$, where $n$ is the number of input items.

*Remark:* Some programmers use a bubble sort algorithm that contains a 1-bit variable FLAG (or a *logical* variable FLAG) to signal when no interchange takes place during a pass. If FLAG = 0 after any pass, then the list is already sorted and there is no need to continue. This may cut down on the number of passes. However, when using such a flag, one must initialize, change and test the variable FLAG during each pass. Hence the use of the flag is efficient only when the list originally is "almost" in sorted order.